

DIGITAISBR

Documentação da Plataforma

Associação de criadores digitais — arquitetura, modelo de dados, regras de negócio, interface, capacidade medida, análise de segurança e guia de publicação.

37

TABELAS

170

ROTAS

40

TELAS

77

TESTES E2E

Sumário

1. Visão geral do produto

O que é a DigitaisBR · As duas áreas · Os três planos · Como o dinheiro circula

2. Arquitetura

Visão em camadas · Stack e escolhas · Estrutura do código · Origem do domínio

3. Autenticação e autorização

Fluxo de login · Tokens e rotação · Os quatro guards · Matriz de permissões

4. Modelo de dados

Mapa das 37 tabelas · Diagrama de relacionamentos · Decisões de modelagem · Enums

5. Regras de negócio

Ciclo de vida da venda · Cálculo da comissão · Cupons · Saques e saldo · Limites por plano

6. Módulos funcionais

Área administrativa · Portal do associado · Área pública

7. Referência da API

Convenções · Paginação, busca e filtros · Erros · As 170 rotas

8. A interface

Tecnologias · As três áreas · 40 telas · Decisões estruturais · Identidade visual

9. Como foi desenvolvido

As cinco etapas · Pipeline reexecutável · O que a conferência revelou

10. Escalabilidade e capacidade

Capacidade medida por endpoint · Comportamento sob concorrência · Quantos usuários simultâneos · Gargalo corrigido · Caminhos de crescimento

11. Análise de segurança

Testes ofensivos executados · Controles implementados · Vulnerabilidade corrigida · Pendências

12. Dados e fidelidade

Pipeline de extração · Conteúdo do seed · Conferência com as telas originais

13. Qualidade e testes

Cobertura da suíte · Como executar

14. Operação

Subir o ambiente · Variáveis · Migrations · Checklist

15. Publicar na internet

Arquitetura de produção · Checklist obrigatório · Passo a passo · Estimativa de infraestrutura

Visão geral do produto

A DigitaisBR é uma plataforma de **associação de criadores digitais**. Ela conecta três partes: os *associados* (influenciadores que recomendam e vendem produtos), os *parceiros* (empresas que oferecem produtos e benefícios) e a *administração*, que cura o catálogo, liquida comissões e sustenta a operação.

O modelo de receita tem duas pernas. A primeira é a **assinatura mensal**: cada associado paga por um plano (Básico, Intermediário ou Avançado), o que gera receita recorrente previsível. A segunda é a **venda por indicação**: cada associado monta uma loja virtual com produtos do catálogo, divulga links rastreáveis, e recebe uma comissão percentual sobre o que vende.

1.1 As duas áreas

O sistema se divide em duas experiências distintas, com autorizações separadas.

ÁREA	QUEM ACESSA	O QUE FAZ
Administrativa	ADMIN	Cadastra associados e produtos, cura o catálogo, acompanha vendas, aprova e liquida comissões, gerencia parceiros e benefícios, publica conteúdo, modera a comunidade, atende o suporte, dispara campanhas e acompanha o financeiro (DRE, fluxo de caixa, MRR).
Portal do associado	ASSOCIADO	Monta e personaliza a própria loja, escolhe produtos do marketplace, cria cupons, gera links de afiliado, acompanha vendas e comissões, solicita saques, resgata benefícios, consome conteúdo, participa da comunidade, abre chamados e pede assessoria jurídica ou contábil.
Pública	SEM LOGIN	Vitrine da loja de cada associado, perfil público do criador e o redirecionador dos links de afiliado.

1.2 Os três planos

O plano é o eixo de monetização e, ao mesmo tempo, o mecanismo de controle de acesso: ele define quantos produtos cabem na loja, quais benefícios e conteúdos ficam liberados, quanto de comissão extra o associado ganha e que nível de suporte recebe.

PLANO	MENSALIDADE	PRODUTOS	COMISSÃO EXTRA	SUPORTE	DESTAQUES
Básico	R\$ 49,90	até 20	—	Email	Benefícios essenciais, conteúdos educacionais, comunidade, loja padrão

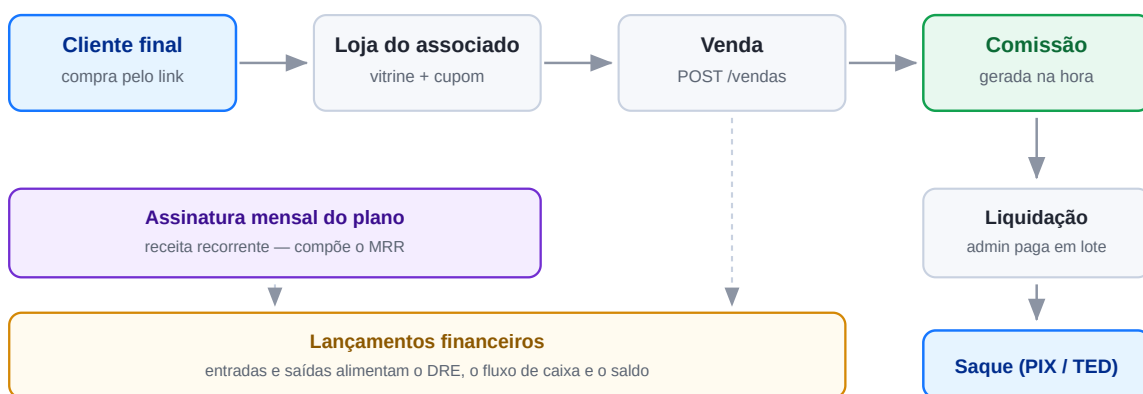
PLANO	MENSALIDADE	PRODUTOS	COMISSÃO EXTRA	SUORTE	DESTAQUES
Intermediário	R\$ 99,90	até 100	+2%	Chat	Tudo do Básico, mais suporte jurídico e contábil, personalização visual da loja e relatórios avançados
Avançado	R\$ 199,90	ilimitado	+5%	Prioritário	Tudo do Intermediário, mais produtos exclusivos, assessoria completa e consultoria mensal

COMO O PLANO GATEIA O ACESSO

Produtos, benefícios e conteúdos carregam um campo `planoMinimo`. Quando um associado do Básico lista o catálogo, os itens acima do plano dele não somem — voltam marcados com `bloqueado: true`, junto com o plano exigido. Isso reproduz as seções “Disponíveis” e “Bloqueados” das telas e cria o gancho natural de upgrade.

1.3 Como o dinheiro circula

O fluxo abaixo é o coração do negócio, e cada seta corresponde a uma operação real da API.

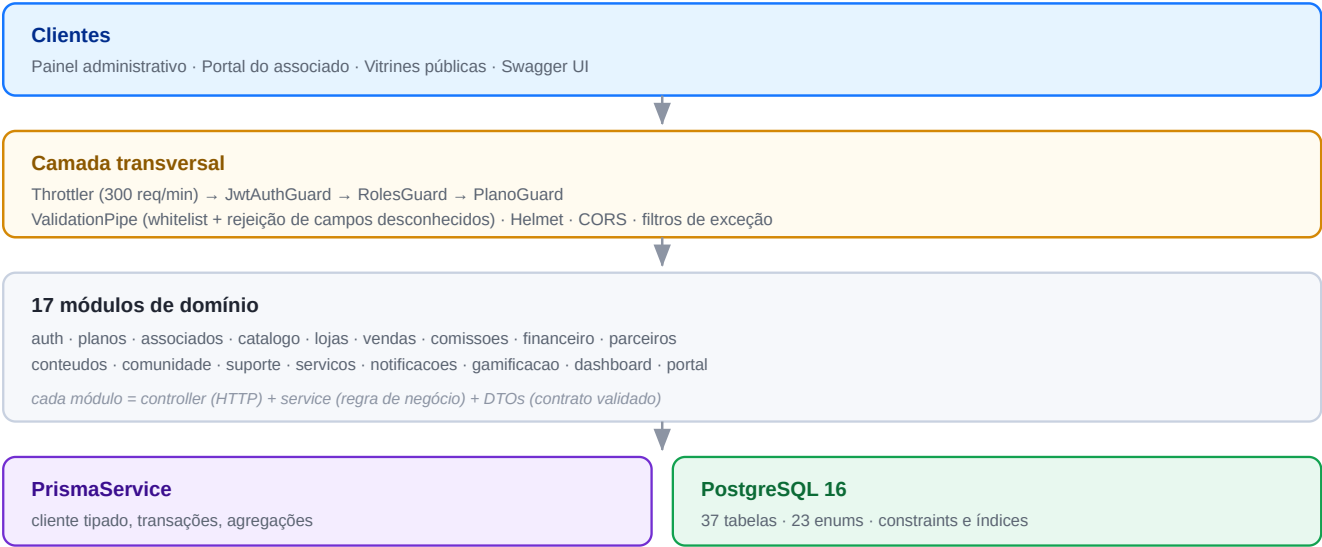


Do clique do cliente ao saque do associado — e o que cada etapa registra no financeiro.

Arquitetura

O backend é uma API REST em NestJS, organizada por domínio de negócio, com PostgreSQL acessado via Prisma. Não há estado em memória: toda a verdade vive no banco, o que permite rodar várias instâncias atrás de um balanceador sem sessão pegajosa.

2.1 Visão em camadas



Toda requisição atravessa a mesma cadeia antes de tocar a regra de negócio.

2.2 Stack e por que cada peça

PEÇA	VERSÃO	MOTIVO DA ESCOLHA
NestJS	11	Modularidade explícita e injeção de dependências. Com 17 domínios, a estrutura imposta pelo framework evita que o projeto vire um punhado de controllers soltos.
Prisma	6	Schema declarativo como fonte única da verdade, tipos gerados a partir dele e migrations versionadas. O <code>groupBy</code> e o <code>aggregate</code> tipados resolvem quase toda a camada de dashboards sem SQL manual.
PostgreSQL	16	Necessário por três recursos usados de fato: <code>Decimal</code> exato para dinheiro, arrays nativos (recursos do plano, especialidades, público-alvo) e enums no banco.
JWT + Passport	—	Autenticação sem estado, com refresh token rotativo guardado hashado no banco.
class-validator	0.14	Validação declarativa nos DTOs, que o Swagger reaproveita para documentar o contrato.

PEÇA	VERSÃO	MOTIVO DA ESCOLHA
Swagger	11	Documentação viva gerada dos próprios decoradores — nunca desatualiza em relação ao código.

2.3 Estrutura do código

```

apps/api/
├─ prisma/
│  ├─ schema.prisma      37 modelos, 23 enums – a fonte da verdade
│  ├─ migrations/        histórico versionado do banco
│  ├─ seed.ts             carrega o dataset real nas tabelas
│  └─ seed-data/*.json    24 arquivos, 374 registros extraídos
├─ src/
│  ├─ main.ts             bootstrap, Swagger, CORS, Helmet, filtros globais
│  ├─ app.module.ts        composição dos módulos e cadeia de guards
│  ├─ common/
│  │  ├─ prisma/          PrismaService (módulo global)
│  │  ├─ decorators/      @Public @Roles @PlanoMinimo @CurrentUser
│  │  ├─ guards/          JwtAuthGuard RolesGuard PlanoGuard
│  │  ├─ filters/         tradução de erros do Prisma para HTTP
│  │  ├─ dto/             PaginationDto e PaginatedResult
│  │  └─ utils/           busca textual, ordenação segura, intervalo de datas
│  └─ modules/<domínio>/
│     ├─ *.controller.ts  rotas, autorização, documentação
│     ├─ *.service.ts      regra de negócio e acesso a dados
│     ├─ *.module.ts       montagem e exportação
│     └─ dto/              contratos de entrada validados
└─ test/                  suíte end-to-end (77 asserções)

```

UMA REGRA DE ORGANIZAÇÃO

Controllers não contêm regra de negócio — apenas roteiam, autorizam e documentam. Toda decisão de domínio vive no service, onde há acesso ao estado real do banco. É por isso que validações como “este produto é exclusivo do plano Avançado” ficam no service e não num guard: o guard não sabe qual produto está sendo pedido.

2.4 De onde veio o domínio

Este backend não foi desenhado a partir de uma especificação escrita. Ele foi **reconstruído por engenharia reversa** de 113 telas capturadas da plataforma original — uma SPA React com Firebase, cujo bundle minificado não permitia recuperar o código-fonte.

O caminho foi: capturar o HTML renderizado de cada rota → extrair as tabelas, os cards e as páginas de detalhe → inferir as entidades, os campos e os relacionamentos → modelar o schema → reimplementar as regras que os dados denunciavam. O script `tools/extract.py` materializa esse elo e continua reexecutável.



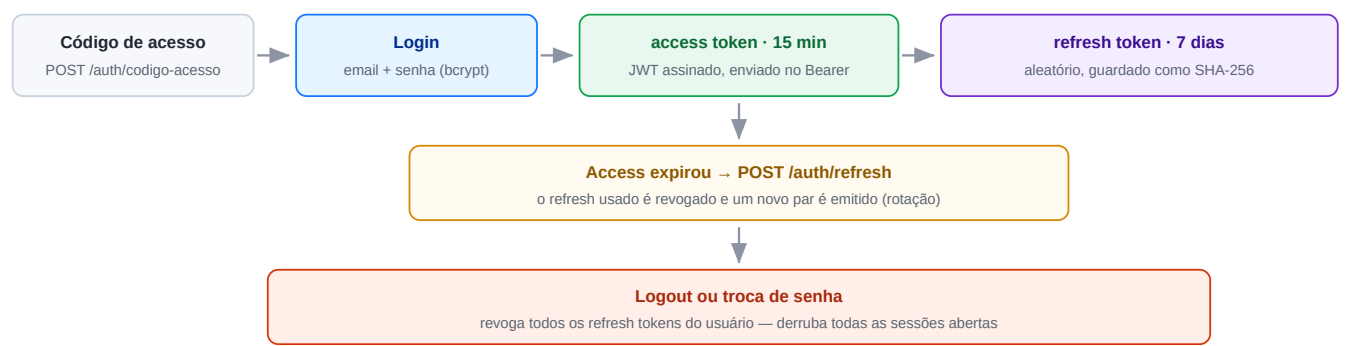
O pipeline que transformou telas capturadas em um banco de dados relacional.

Autenticação e autorização

A segurança se apoia em quatro camadas independentes, aplicadas em cadeia a toda requisição. Cada uma responde a uma pergunta diferente: quantas requisições, quem é, que papel tem, e qual plano assina.

3.1 Fluxo de autenticação

A plataforma original protegia o acesso com um código antes mesmo do login. Esse gate foi preservado como `POST /auth/codigo-acesso`, e o fluxo completo é:



Ciclo de vida das credenciais, do gate de acesso à revogação.

Decisões de segurança tomadas

DECISÃO	CONSEQUÊNCIA PRÁTICA
Senha com bcrypt (10 rounds)	Vazamento do banco não expõe senhas em texto claro.
Refresh token guardado hasheado	Quem lê a tabela <code>refresh_tokens</code> não consegue se passar por um usuário.
Rotação a cada renovação	Um refresh token roubado só serve uma vez; o uso legítimo seguinte falha e denuncia o roubo.
Access token curto (15 min)	Reduz a janela de exploração de um token interceptado.
Validação do usuário a cada requisição	Desativar uma conta tem efeito imediato, sem esperar o token expirar.
Troca de senha revoga sessões	Recuperar a conta após um comprometimento realmente expulsa o invasor.
<code>forbidNonWhitelisted</code> no <code>ValidationPipe</code>	Campo não declarado no DTO gera <code>400</code> — bloqueia <i>mass assignment</i> .

3.2 Os quatro guards

Registrados globalmente, executam nesta ordem para toda requisição:

GUARD	PERGUNTA QUE RESPONDE	COMO SE CONFIGURA
ThrottlerGuard	Excedeu o limite?	300 requisições por minuto, por IP. Devolve <code>429</code> .
JwtAuthGuard	Quem é você?	Exige <code>Authorization: Bearer</code> . Rotas com <code>@Public()</code> passam direto.
RolesGuard	Tem o papel certo?	<code>@Roles(Role.ADMIN)</code> no controller ou no método.
PlanoGuard	O plano cobre isso?	<code>@PlanoMinimo(NivelPlano.INTERMEDIARIO)</code> . Admin sempre passa.

ONDE OS GUARDS NÃO BASTAM

Guards decidem por metadados estáticos da rota. Regras que dependem do *dado sendo manipulado* — “este produto específico é exclusivo do Avançado”, “esta loja já atingiu o limite de 20 produtos do Básico”, “este chamado pertence a outro associado” — são validadas no service, que tem acesso ao estado. Ignorar essa distinção é como se abrem brechas de autorização horizontal.

3.3 Matriz de permissões

RECURSO	PÚBLICO	ASSOCIADO	ADMINISTRADOR
Vitrine da loja · perfil público · redirecionador de link	✓ leitura	✓	✓
Catálogo de produtos	—	✓ com bloqueio por plano	✓ total + escrita
Benefícios e conteúdos	—	✓ com bloqueio por plano	✓ total + escrita
Própria loja, cupons, links, métricas	—	✓ escrita	✓
Comunidade	—	✓ publicar, curtir, comentar	✓ + fixar e moderar
Chamados de suporte	—	✓ só os próprios, sem notas internas	✓ todos + notas internas
Saques	—	✓ solicitar do próprio saldo	✓ aprovar, processar, rejeitar
Cadastro de associados e produtos	—	—	✓
Vendas, comissões, financeiro	—	— (vê os próprios via portal)	✓
Campanhas, parceiros, relatórios	—	—	✓

Modelo de dados

São 37 tabelas e 23 enums, agrupadas em oito domínios. A maior parte das regras de consistência foi empurrada para o banco — chaves estrangeiras, índices únicos e enums — de modo que um bug na aplicação não consiga produzir um estado impossível.

4.1 Mapa das tabelas

Autenticação e auditoria

TABELA	CAMPOS QUE IMPORTAM	PAPEL
usuarios	email (único), senhaHash, role, ativo, ultimoLogin	Credencial de acesso. Um usuário é ADMIN ou ASSOCIADO.
refresh_tokens	tokenHash (único), expiraEm, revogado	Sessões abertas. Permite revogação seletiva ou em massa.
logs_auditoria	acao, entidade, entidadeId, dados (JSON), ip	Rastro de operações sensíveis. Sobrevive à exclusão do usuário.

Associados e planos

TABELA	CAMPOS QUE IMPORTAM	PAPEL
planos	nivel (único), preco, recursos (array), limiteProdutos, comissaoExtraPct	Define preço e o que cada nível libera. É referenciado como plano mínimo em outras tabelas.
associados	handle (único), cpfCnpj (único), nicho, seguidores, engajamento, status, pontuacao	O criador de conteúdo. Relaciona-se 1:1 com usuário e 1:1 com loja.
assinaturas	valor, inicioEm, fimEm, ativa	Histórico de planos. Só as ativas compõem o MRR.
redes_sociais_contas	(associado, rede) (único), handle, seguidores, engajamento	Perfis conectados e suas métricas.

Catálogo e lojas

TABELA	CAMPOS QUE IMPORTAM	PAPEL
categorias_produto	nome (único), slug (único), cor	Taxonomia do catálogo.
produtos	sku (único), preco, comissaoPct, estoque, planoMinimold, status	Item vendável. <code>estoque = -1</code> significa ilimitado.
lojas	associadold (único), slug (único), corPrimaria, ativa, visualizacoes	Vitrine pública do associado.
loja_produtos	(loja, produto) (único), destaque, ordem	Curadoria: quais produtos o associado escolheu exibir e em que ordem.

Vendas e comissões

TABELA	CAMPOS QUE IMPORTAM	PAPEL
<code>vendas</code>	ref <único>, quantidade, precoUnitario, desconto, total, status, dataVenda	Transação. Guarda o preço praticado no momento — mudar o produto depois não reescreve a história.
<code>comissoes</code>	vendald <único>, percentual, valor, status, pagoEm, saqueId	Direito do associado sobre a venda. Relação 1:1 com a venda.
<code>cupons</code>	(associado, codigo) <único>, tipoDesconto, desconto, compraMinima, usos, limiteUsos	Desconto criado pelo associado. O contador de usos é incrementado na venda.
<code>links_afiliado</code>	codigo <único>, cliques, conversoes, receita, comissao	Link rastreável por produto, com métricas acumuladas.
<code>metricas_diarias</code>	(associado, data) <único>, cliques, conversoes, receita, comissao	Série pré-agregada. Evita varrer eventos brutos para montar gráficos.

Parceiros e benefícios

TABELA	CAMPOS QUE IMPORTAM	PAPEL
<code>parceiros</code>	nome <único>, cnpj <único>, segmento, ativo	Empresa conveniada.
<code>beneficios</code>	nome <único>, tipo, valorLabel, planoMinimold, utilizacoes	Vantagem oferecida: desconto, acesso, cashback ou serviço.
<code>beneficios_usos</code>	beneficiold, associadold, usadoEm	Resgates, para medir adesão.

Conteúdo e comunidade

TABELA	CAMPOS QUE IMPORTAM	PAPEL
<code>conteudos</code>	slug <único>, tipo, status, planoMinimold, visualizacoes, curtidas	Material educativo: artigo, vídeo, podcast, curso ou e-book.
<code>conteudos_interacoes</code>	(conteudo, associado) <único>, visualizado, curtido	Garante que a visualização conte uma vez por pessoa.
<code>materiais_divulgacao</code>	tipo, dimensao, arquivoUrl, textoCopy, downloads	Banners, stories, posts, vídeos e textos prontos para o associado divulgar.
<code>categorias_comunidade</code>	nome <único>, slug <único>, icone	Seções do fórum.
<code>posts_comunidade</code>	autorId, categoriald, conteudo, fixado, visualizacoes	Publicação no feed.
<code>comentarios_post</code>	postId, autorId, conteudo	Respostas.
<code>curtidas_post</code>	(post, associado) <único>	O índice único torna impossível curtir duas vezes — regra no banco, não no código.

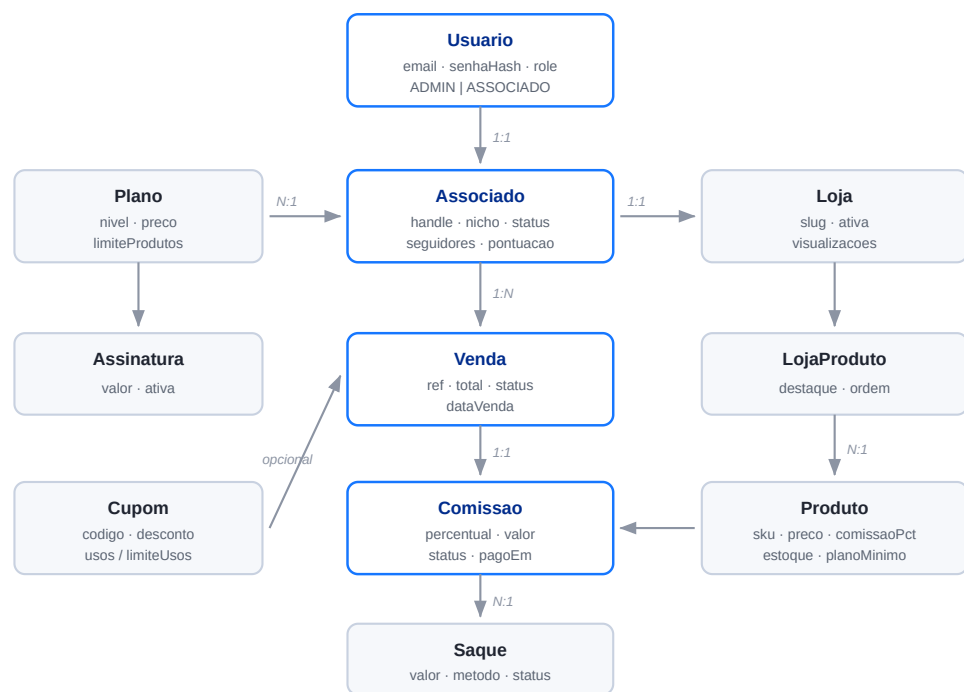
Suporte e serviços

TABELA	CAMPOS QUE IMPORTAM	PAPEL
<code>tickets</code>	numero (único), categoria, prioridade, status, atribuidoAId, resolvidoEm	Chamado de atendimento.
<code>tickets_mensagens</code>	ticketId, autorId, conteudo, interna	Thread da conversa. <code>interna = true</code> nunca chega ao associado.
<code>escritorios_parceiros</code>	nome (único), tipo, especialidades (array), atendimentos	Banca jurídica ou contábil conveniada.
<code>profissionais</code>	nome, especialidade, nota, avaliacoes, valorHora, disponivel	Advogado ou contador atendendo.
<code>solicitacoes_servico</code>	assunto, status, profissionalId, concluidaEm	Pedido de assessoria feito pelo associado.

Comunicação, financeiro e gamificação

TABELA	CAMPOS QUE IMPORTAM	PAPEL
<code>notificacoes</code>	associadoId (anulável), tipo, canal, lida, lidaEm	<code>associadoId = null</code> representa um comunicado para todos.
<code>campanhas</code>	codigo (único), publicoAlvo (array de planos), enviados, aberturas, status	Disparo em massa segmentado por plano.
<code>saques</code>	valor, metodo, destino, status, concluidoEm, motivoRejeicao	Retirada de comissões.
<code>lancamentos_financeiros</code>	tipo, categoria, valor, competencia	Livro-caixa. Sustenta DRE, fluxo de caixa e saldo sem recalcular vendas.
<code>conquistas</code>	nome (único), meta, pontos, ordem	Catálogo de metas gamificadas.
<code>conquistas_associados</code>	(conquista, associado) (único), progresso, desbloqueada	Progresso individual.

4.2 Relacionamentos centrais



Núcleo transacional. A comissão é 1:1 com a venda e agrupa-se num saque quando liquidada.

4.3 Decisões de modelagem

DECISÃO	JUSTIFICATIVA
<code>Decimal(10,2)</code> e <code>Decimal(12,2)</code> para dinheiro	Ponto flutuante acumula erro em soma de moeda. Numa base com milhares de comissões, o desvio aparece no fechamento. O tipo decimal do Postgres é exato.
<code>estoque = -1</code> significa ilimitado	Preserva a semântica da plataforma original e evita uma coluna booleana extra que poderia divergir do valor numérico.
<code>Comissao</code> em 1:1 com <code>Venda</code>	A comissão nasce com a venda e acompanha o destino dela. Manter tabela separada permite ter status e data de pagamento próprios, sem inchar a venda.
<code>precoUnitario</code> gravado na venda	O preço do produto muda com o tempo. Congelar o valor praticado mantém o histórico financeiro correto para sempre.
<code>Notificacao.associadoId</code> anulável	<code>null</code> é broadcast. Evita criar 40 mil linhas para um comunicado geral.
<code>LancamentoFinanceiro</code> como livro-caixa separado	DRE e fluxo de caixa consultam uma tabela pequena e indexada por competência, em vez de reagregar todas as vendas e comissões a cada carregamento de tela.
<code>MetricaDiaria</code> única por (associado, dia)	Gráficos de performance leem uma linha por dia. A alternativa — varrer cliques brutos — degradaria à medida que o tráfego crescesse.
<code>planoMinimoId</code> em produto, benefício e conteúdo	A regra de acesso vive no dado. Criar um produto exclusivo é preencher um campo, não alterar código.

DECISÃO	JUSTIFICATIVA
Índices únicos compostos em curtidas, interações e cupons	Transferem a regra de unicidade para o banco. Nem uma condição de corrida consegue gerar duas curtidas da mesma pessoa no mesmo post.
<code>onDelete</code> explícito em cada relação	<code>Cascade</code> onde o filho não faz sentido sozinho (loja de um associado); <code>SetNull</code> onde o registro deve sobreviver (autor de um log, atendente de um ticket encerrado).

4.4 Enums do domínio

Vinte e três enums transferem para o banco a consistência de status e tipos. Os principais:

ENUM	VALORES
<code>Role</code>	ADMIN · ASSOCIADO
<code>NivelPlano</code>	BASICO · INTERMEDIARIO · AVANÇADO
<code>StatusAssociado</code>	ATIVO · INATIVO · SUSPENSO
<code>StatusProduto</code>	ATIVO · INATIVO · ESGOTADO
<code>StatusVenda</code>	AGUARDANDO_PGTO · PAGA · CANCELADA · REEMBOLSADA
<code>StatusComissao</code>	AGUARDANDO_PGTO · PROCESSANDO · PAGA · CANCELADA
<code>TipoBeneficio</code>	DESCONTO · ACESSO · CASHBACK · SERVICO
<code>TipoConteudo</code>	ARTIGO · VIDEO · PODCAST · CURSO · EBOOK
<code>StatusTicket</code>	ABERTO · EM_ANDAMENTO · RESOLVIDO · FECHADO
<code>PrioridadeTicket</code>	BAIXA · MEDIA · ALTA · URGENTE
<code>TipoNotificacao</code>	SISTEMA · VENDA · COMISSAO · BENEFICIO · SUPORTE · CONTEUDO
<code>CanalNotificacao</code>	IN_APP · EMAIL · PUSH
<code>MetodoSaque</code> · <code>StatusSaque</code>	PIX · TED / PENDENTE · PROCESSANDO · CONCLUIDO · REJEITADO
<code>TipoLancamento</code> · <code>CategoriaLancamento</code>	ENTRADA · SAIDA / ASSINATURA · VENDA · COMISSAO · INFRAESTRUTURA · MARKETING · SUPORTE · JURIDICO · OUTROS
<code>RedeSocial</code>	INSTAGRAM · YOUTUBE · TIKTOK · TWITTER · FACEBOOK · LINKEDIN

Regras de negócio

Este capítulo descreve o que a API faz além de guardar e devolver registros. São as regras que, se implementadas errado, produzem prejuízo financeiro ou inconsistência contábil.

5.1 Registro de uma venda

`POST /vendas` executa oito passos dentro de **uma única transação**. Se qualquer um falhar, nada é gravado — não existe estado meio-feito em que o estoque baixou mas a comissão não foi criada.

1. **Valida o produto** — precisa existir e estar `ATIVO`; senão, `409`.
2. **Calcula o bruto** — `preço × quantidade`.
3. **Aplica o cupom**, se houver. Confere quatro condições: pertence ao associado, está ativo, não expirou, não estourou o limite de usos, e a compra atinge o mínimo exigido. Incrementa o contador de usos.
4. **Fecha o total** — `bruto - desconto`, arredondado a dois decimais.
5. **Calcula a comissão** — percentual do produto *somado* ao bônus do plano.
6. **Baixa o estoque** — se finito; se zerar, o produto vira `ESGOTADO`.
7. **Gera a referência** — `CHK-NNNNN`, sequencial e contínuo com o histórico.
8. **Cria venda + comissão e notifica** o associado.

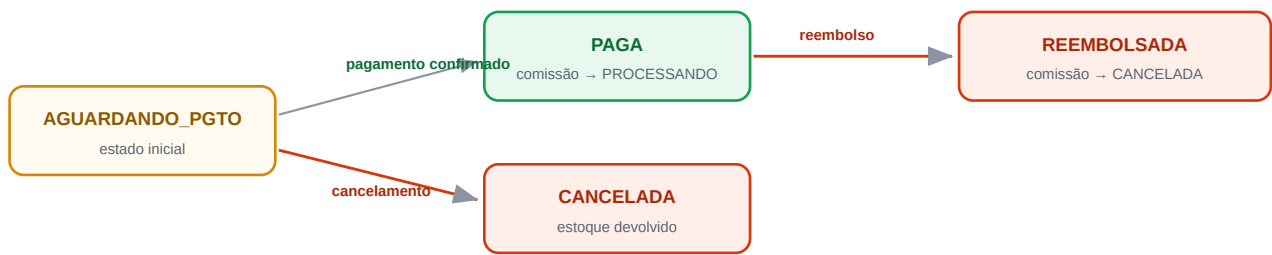
A fórmula da comissão

```
// percentual efetivo = o do produto + o bônus do plano do associado
percentual = produto.comissaoPct + associado.plano.comissaoExtraPct
valor      = total × percentual / 100
```

Um exemplo concreto: um produto de R\$ 149,90 com 18% de comissão, vendido em duas unidades por um associado do plano Básico (bônus 0%), gera $299,80 \times 18\% = \text{R\$ } 53,96$. O mesmo produto vendido por um associado Avançado (bônus +5%) renderia $299,80 \times 23\% = \text{R\$ } 68,95$. O plano se paga pelo próprio volume.

5.2 Ciclo de vida da venda

O status da venda é uma máquina de estados fechada. Transições fora do diagrama devolvem `409 Conflict` com a lista do que seria permitido — o que impede, por exemplo, reembolsar uma venda que nunca foi paga.



CANCELADA e REEMBOLSADA são estados finais — nenhuma transição sai deles

Máquina de estados da venda e os efeitos colaterais de cada transição.

Cada transição propaga consequências, que também correm em transação:

TRANSIÇÃO	COMISSÃO	ESTOQUE	CONTADOR DO PRODUTO
→ PAGA	vai a PROCESSANDO	permanece baixado	incrementa
→ CANCELADA	vai a CANCELADA	devolvido; se estava esgotado, reativa	—
→ REEMBOLSADA	vai a CANCELADA	devolvido; se estava esgotado, reativa	decrementa

5.3 Comissões e liquidação

A comissão percorre quatro estados. Ela nasce `AGUARDANDO_PGTO` junto com a venda, vai a `PROCESSANDO` quando a venda é paga, e só chega a `PAGA` quando o administrador a liquida — individualmente ou em lote.

O pagamento em lote (`POST /comissoes/pagar`) recebe uma lista de identificadores e, antes de qualquer escrita, verifica que todas existem e todas estão em estado liquidável. Se uma única já estiver paga, o lote inteiro é recusado com `409` — evitando pagamento duplicado por clique repetido. Ao final, consolida os valores por associado e envia **uma** notificação por pessoa, com o total, em vez de uma por comissão.

5.4 Saldo e saques

O saldo disponível não é um campo guardado — seria uma fonte constante de divergência. Ele é calculado no momento da consulta:

```

disponivel = comissoes(PAGA)
            - saques(CONCLUIDO)
            - saques(PENDENTE + PROCESSANDO)  ← reserva o que já foi pedido
  
```

Subtrair os saques em andamento é o que impede o saque duplo: pedir R\$ 500 duas vezes seguidas falha na segunda tentativa, porque o primeiro pedido já reservou o valor.

ESTADO DO SAQUE	TRANSIÇÕES POSSÍVEIS
PENDENTE	→ PROCESSANDO · → REJEITADO (<i>exige motivo</i>)

ESTADO DO SAQUE	TRANSIÇÕES POSSÍVEIS
PROCESSANDO	→ CONCLUÍDO · → REJEITADO (<i>exige motivo</i>)
CONCLUÍDO	estado final — gera lançamento de saída no caixa
REJEITADO	estado final

5.5 Limites e exclusividades por plano

REGRA	ONDE É CHECADA	COMPORTAMENTO
Limite de produtos na loja	<code>LojasService</code>	Básico até 20, Intermediário até 100, Avançado ilimitado. Estourar devolve <code>409</code> sugerindo upgrade.
Produto exclusivo de plano	<code>LojasService</code>	Adicionar produto acima do plano devolve <code>403</code> nomeando o plano exigido.
Personalização visual da loja	<code>PortalService</code>	Cor, banner e logo exigem Intermediário. Renomear e descrever é livre.
Benefício acima do plano	<code>ParceirosService</code>	Resgatar devolve <code>403</code> ; listar devolve com <code>bloqueado: true</code> .
Conteúdo acima do plano	<code>ConteudosService</code>	Abrir devolve <code>403</code> ; listar marca como bloqueado.
Rotas com <code>@PlanoMinimo</code>	<code>PlanoGuard</code>	Bloqueio no nível da rota, antes de tocar o service.

5.6 Suporte: isolamento e notas internas

Duas regras de confidencialidade governam o módulo de atendimento, e ambas são testadas na suíte end-to-end:

- **Isolamento horizontal** — um associado que tente abrir o chamado de outro recebe `403` , mesmo conhecendo o identificador. A checagem compara o dono do ticket com o associado autenticado.
- **Notas internas** — mensagens com `interna: true` são filtradas da resposta quando quem consulta não é administrador. Um associado tentando criar uma recebe `403` .

Além disso, a primeira resposta do atendimento move o ticket de `ABERTO` para `EM_ANDAMENTO` automaticamente, e apenas mensagens públicas geram notificação para o associado — uma nota interna não avisa ninguém de fora.

5.7 Gamificação

A pontuação do ranking é derivada dos dados reais, não digitada: `vendas × 100 + comissões pagas acumuladas` . O endpoint de recálculo reprocessa também o progresso das conquistas, medindo cada uma contra a métrica que lhe corresponde — volume de vendas, comissão acumulada, produtos na loja, cupons criados ou posts publicados.

Módulos funcionais

Dezessete módulos, cada um responsável por um domínio. Abaixo, o que cada um entrega e as operações que vão além do CRUD.

6.1 Área administrativa

Dashboard e relatórios

Consolida em uma chamada os cartões da home: associados ativos, vendas aprovadas, receita, lojas ativas, comissões pagas e pendentes, saldo financeiro e produtos ativos. Fornece ainda a série mensal de receita × comissões (SQL agregado direto, não N+1) e o relatório analítico com vendas por status, associados por plano, e os cinco produtos e associados de maior receita — tudo filtrável por período.

Associados

Cadastro completo com busca textual, filtros por status, plano, nicho e faixa de seguidores, e ordenação por qualquer campo permitido. Criar um associado é uma operação composta: gera o usuário com senha (retornada uma única vez se não informada), a assinatura do plano e a loja virtual — tudo de uma vez.

Duas operações merecem nota. **Trocar de plano** encerra a assinatura vigente, abre uma nova e notifica o associado, em transação. **Alterar status** propaga: inativar um associado também desativa o acesso dele e tira a loja do ar. E a **exclusão** é recusada com `409` se houver vendas registradas — histórico financeiro não se apaga; inativa-se.

Catálogo

Produtos e categorias. Cada produto responde com `ganhoEstimado` já calculado e, para associados, o campo `bloqueado`. Os filtros cobrem faixa de preço, comissão mínima, categoria, status e exclusividade. Remover um produto com vendas é recusado — a orientação é inativá-lo, preservando o histórico.

Lojas

Gestão das vitrines: curadoria de produtos com destaque e ordenação, personalização visual e ativação. O endpoint de detalhe traz desempenho calculado — vendas aprovadas, receita, visualizações e taxa de conversão.

Vendas

Registro transacional (seção 5.1), máquina de estados (5.2), indicadores do topo da tela e **exportação CSV** da listagem filtrada, com escape correto de aspas — o equivalente ao botão “Exportar CSV” da tela original, agora funcional.

Comissões

Listagem com filtros por status, associado e período; liquidação em lote com validação prévia; consulta de saldo por associado; e ranking dos que mais receberam.

Financeiro

A área mais analítica. Entrega visão geral (saldo, MRR, ARR, margem líquida, comissões a pagar), fluxo de caixa mensal, DRE com composição percentual por categoria, projeção de MRR baseada no crescimento observado nos últimos 30 dias (limitada a 20% ao mês para não extrapolar), gestão de saques e extrato consolidado por associado.

Parceiros, benefícios e conteúdos

Cadastro de parceiros com CNPJ validado por formato, benefícios tipados (desconto, acesso, cashback, serviço) com plano mínimo e contador de utilizações, e uma biblioteca de conteúdos com controle de publicação. Materiais de divulgação — banners, stories, posts, vídeos e copies — com contagem de downloads.

Comunidade, suporte e serviços

Feed com categorias, fixação de posts, curtidas, comentários e moderação. Central de atendimento com thread, prioridade, atribuição e notas internas. Rede de escritórios jurídicos e contábeis com profissionais, avaliações e solicitações de atendimento.

Notificações e campanhas

Notificações individuais ou em broadcast, com tipo e canal. Campanhas segmentadas por plano, com estados de rascunho, agendada, enviada e cancelada; o disparo cria uma notificação por destinatário e registra a contagem de envios.

6.2 Portal do associado

ÁREA	O QUE O ASSOCIADO FAZ
Painel	Vê vendas, receita, comissões recebidas e pendentes, uso do plano e as vendas recentes.
Perfil	Edita bio, nicho e contato; controla se email e telefone aparecem no perfil público.
Meu plano	Vê o consumo (produtos, benefícios, conteúdos), o ROI da mensalidade e exatamente o que o próximo plano desbloquearia — número de benefícios, conteúdos e comissão extra.
Minha loja	Escolhe produtos do marketplace, ordena, destaca e personaliza a vitrine.
Cupons	Cria códigos de desconto com validade, limite de usos e compra mínima. Cupom já utilizado não pode ser excluído — apenas desativado.
Links de afiliado	Gera links rastreáveis por produto e acompanha cliques, conversões, taxa e receita.

ÁREA	O QUE O ASSOCIADO FAZ
Performance	Série diária de cliques e conversões, com os produtos de maior receita.
Financeiro	Consulta saldo e solicita saque por PIX ou TED.
Benefícios e conteúdos	Resgata benefícios e consome material educativo, dentro do que o plano libera.
Comunidade e suporte	Publica, comenta, curte; abre e acompanha os próprios chamados.
Ranking	Vê a própria posição, o percentil e o progresso das conquistas.
Redes sociais	Conecta perfis e acompanha seguidores, engajamento e volume de posts.

6.3 Área pública

ROTA	COMPORTAMENTO
<code>GET /lojas/publica/:slug</code>	Vitrine da loja. Devolve apenas produtos ativos, contabiliza uma visualização por acesso e recusa lojas desativadas com <code>403</code> .
<code>GET /associados/perfil/:handle</code>	Perfil público do criador, respeitando as preferências de privacidade: email e telefone só aparecem se o associado permitiu.
<code>GET /portal/r/:codigo</code>	Redirecionador do link de afiliado. Registra o clique, soma na métrica diária e devolve o destino.

Referência da API

170 rotas em 17 módulos. Esta referência lista todas, agrupadas por área. A versão navegável, com schemas de entrada e saída e formulário de teste, está no Swagger em `/api/docs`.

7.1 Convenções

ITEM	CONVENÇÃO
URL base	<code>http://localhost:3000/api</code>
Autenticação	<code>Authorization: Bearer <access token></code>
Formato	JSON em requisição e resposta; UTF-8
Valores monetários	número com dois decimais (ex.: <code>149.9</code>), nunca string
Datas	ISO 8601 na resposta; <code>YYYY-MM-DD</code> aceito em filtros
Identificadores	UUID v4; algumas rotas aceitam também o código natural (<code>CHK-10056</code> , <code>SUP-001</code> , SKU, slug)

7.2 Paginação, busca, filtro e ordenação

Toda listagem aceita os mesmos parâmetros e devolve a mesma envelopagem:

PARÂMETRO	PADRÃO	EFEITO
<code>page</code>	1	Página, começando em 1
<code>limit</code>	10	Itens por página; máximo 100 — acima disso, <code>400</code>
<code>search</code>	—	Busca textual, sem distinção de maiúsculas, nos campos relevantes do recurso
<code>sort</code>	—	Campo de ordenação; só nomes de uma lista permitida por recurso
<code>order</code>	desc	<code>asc</code> ou <code>desc</code>
<code>de / ate</code>	—	Recorte de período nos recursos com data

```
GET /api/associados?
search=ana&status=ATIVO&plano=AVANÇADO&sort=seguidores&order=desc&limit=20

{
  "data": [ ... ],
  "meta": {
    "total": 40, "page": 1, "limit": 20,
    "totalPages": 2, "hasNext": true, "hasPrev": false
  }
}
```

ORDENAÇÃO SEGURA

O parâmetro `sort` é confrontado com uma lista branca por recurso antes de virar cláusula `ORDER BY`. Um nome fora da lista cai silenciosamente no padrão, em vez de vaziar estrutura interna ou permitir injeção.

7.3 Erros

Toda resposta de erro segue o mesmo formato, e os erros do Prisma são traduzidos para mensagens em português antes de chegar ao cliente.

CÓDIGO	QUANDO OCORRE	EXEMPLO DE MENSAGEM
400	Entrada inválida	"A senha deve ter ao menos 8 caracteres."
401	Sem token, token expirado ou credenciais erradas	"Email ou senha inválidos."
403	Papel ou plano insuficiente, ou recurso de outro associado	"Recurso disponível a partir do plano AVANÇADO."
404	Registro inexistente (<i>P2025</i>)	"Registro não encontrado."
409	Conflito de estado ou violação de unicidade (<i>P2002</i>)	"Transição inválida: PAGA → AGUARDANDO_PGTO."
429	Acima de 300 requisições por minuto	"ThrottlerException: Too Many Requests"

```
{
  "statusCode": 409,
  "path": "/api/vendas/.../status",
  "timestamp": "2026-08-27T21:14:02.881Z",
  "message": "Transição inválida: AGUARDANDO_PGTO → REEMBOLSADA. Permitidas: PAGA, CANCELADA."
}
```

7.4 As rotas

Legenda: **ADMIN** exige papel de administrador · **PÚBLICO** dispensa autenticação · as demais exigem apenas estar autenticado.

Autenticação · 7 rotas

MÉTODO	CAMINHO	ACESSO	DESCRIÇÃO
POST	/api/auth/alterar-senha	autenticado	Altera a senha e encerra as sessões abertas
POST	/api/auth/codigo-acesso	PÚBLICO	Valida o código de acesso da plataforma (gate anterior ao login)
POST	/api/auth/login	PÚBLICO	Autentica e devolve access + refresh token
POST	/api/auth/logout	autenticado	Revoga todos os refresh tokens do usuário
GET	/api/auth/perfil	autenticado	Dados do usuário autenticado
POST	/api/auth/refresh	PÚBLICO	Renova o access token (com rotação do refresh)
POST	/api/auth/registrar	PÚBLICO	Cria uma conta de associado (com loja e assinatura)

Dashboard e Relatórios · 5 rotas

MÉTODO	CAMINHO	ACESSO	DESCRIÇÃO
GET	/api/dashboard/admin	ADMIN	Cards e totais da home administrativa
GET	/api/dashboard/portal	autenticado	Painel inicial do portal do associado logado
GET	/api/dashboard/receita-comissoes	ADMIN	Série mensal de receita e comissões
GET	/api/dashboard/suporte	ADMIN	Resumo de tickets por status e prioridade
GET	/api/relatorios	ADMIN	Relatório analítico consolidado

Planos · 5 rotas

MÉTODO	CAMINHO	ACESSO	DESCRIÇÃO
GET	/api/planos	autenticado	Lista os planos com contagem de associados ativos
POST	/api/planos	ADMIN	Cria um plano
GET	/api/planos/metricas	ADMIN	MRR, ARR e distribuição de associados por plano
GET	/api/planos/{id}	autenticado	Detalha um plano
PATCH	/api/planos/{id}	ADMIN	Atualiza um plano

Associados · 12 rotas

MÉTODO	CAMINHO	ACESSO	DESCRIÇÃO
GET	/api/associados	ADMIN	Lista associados com busca, filtros e paginação

MÉTODO	CAMINHO	ACESSO	DESCRIÇÃO
POST	/api/associados	ADMIN	Cadastra um associado (cria usuário, assinatura e loja)
GET	/api/associados/estatisticas	ADMIN	Totais por status, plano e nicho
GET	/api/associados/perfil/{handle}	PÚBLICO	Perfil público do associado (respeita as preferências de privacidade)
GET	/api/associados/ranking	autenticado	Ranking de associados por pontuação
DELETE	/api/associados/{id}	ADMIN	Remove um associado sem vendas registradas
GET	/api/associados/{id}	ADMIN	Ficha completa do associado
PATCH	/api/associados/{id}	ADMIN	Atualiza os dados cadastrais
PATCH	/api/associados/{id}/plano	ADMIN	Troca o plano e reabre a assinatura
POST	/api/associados/{id}/redes-sociais	ADMIN	Vincula ou atualiza uma rede social
DELETE	/api/associados/{id}/redes-sociais/{rede}	ADMIN	Desvincula uma rede social
PATCH	/api/associados/{id}/status/{status}	ADMIN	Ativa, inativa ou suspende (reflete no acesso e na loja)

Catálogo · 11 rotas

MÉTODO	CAMINHO	ACESSO	DESCRIÇÃO
GET	/api/catalogo/categorias	autenticado	Lista as categorias com contagem de produtos
POST	/api/catalogo/categorias	ADMIN	Cria uma categoria
DELETE	/api/catalogo/categorias/{id}	ADMIN	Remove uma categoria sem produtos
PATCH	/api/catalogo/categorias/{id}	ADMIN	Atualiza uma categoria
GET	/api/catalogo/produtos	autenticado	Lista o catálogo
POST	/api/catalogo/produtos	ADMIN	Cadastra um produto
GET	/api/catalogo/produtos/estatisticas	ADMIN	Totais por status e categoria, preço e comissão médios
GET	/api/catalogo/produtos/mais-vendidos	autenticado	Top produtos por receita
DELETE	/api/catalogo/produtos/{id}	ADMIN	Remove um produto sem vendas
GET	/api/catalogo/produtos/{id}	autenticado	Detalha um produto por ID ou SKU
PATCH	/api/catalogo/produtos/{id}	ADMIN	Atualiza um produto

Lojas · 9 rotas

MÉTODO	CAMINHO	ACESSO	DESCRIÇÃO
GET	/api/lojas	ADMIN	Lista as lojas com filtros e paginação
POST	/api/lojas	ADMIN	Cria uma loja

MÉTODO	CAMINHO	ACESSO	DESCRIÇÃO
GET	/api/lojas/estatisticas	ADMIN	Totais, visualizações e média de produtos por loja
GET	/api/lojas/publica/{slug}	PÚBLICO	Vitrine pública da loja
GET	/api/lojas/{id}	ADMIN	Detalha uma loja com vitrine e desempenho
PATCH	/api/lojas/{id}	ADMIN	Atualiza dados e personalização da loja
PATCH	/api/lojas/{id}/ativa/{valor}	ADMIN	Ativa ou desativa a loja
POST	/api/lojas/{id}/produtos	ADMIN	Adiciona um produto à vitrine
DELETE	/api/lojas/{id}/produtos/{produtoId}	ADMIN	Remove um produto da vitrine

Vendas · 7 rotas

MÉTODO	CAMINHO	ACESSO	DESCRIÇÃO
GET	/api/vendas	ADMIN	Lista vendas com busca, filtros, período e paginação
POST	/api/vendas	ADMIN	Registra uma venda
GET	/api/vendas/estatisticas	ADMIN	Receita, ticket médio, conversão e cancelamento
GET	/api/vendas/exportar	ADMIN	Exporta a listagem filtrada em CSV
GET	/api/vendas/serie-mensal	ADMIN	Série mensal de receita e comissões
GET	/api/vendas/{id}	ADMIN	Detalha uma venda por ID ou referência (CHK-...)
PATCH	/api/vendas/{id}/status	ADMIN	Altera o status da venda

Comissões · 7 rotas

MÉTODO	CAMINHO	ACESSO	DESCRIÇÃO
GET	/api/comissoes	ADMIN	Lista comissões com filtros de status, associado e período
GET	/api/comissoes/estatisticas	ADMIN	Totais por status, pagas, pendentes e percentual médio
POST	/api/comissoes/pagar	ADMIN	Liquida um lote de comissões
GET	/api/comissoes/saldo/{associadoId}	ADMIN	Saldo disponível para saque de um associado
GET	/api/comissoes/top-associados	ADMIN	Associados que mais receberam comissão
GET	/api/comissoes/{id}	ADMIN	Detalha uma comissão
PATCH	/api/comissoes/{id}/status	ADMIN	Altera o status de uma comissão

Financeiro · 11 rotas

MÉTODO	CAMINHO	ACESSO	DESCRIÇÃO
GET	/api/financeiro/dre	ADMIN	Demonstrativo de resultado com composição por categoria

MÉTODO	CAMINHO	ACESSO	DESCRIÇÃO
GET	/api/financeiro/extrato/{associadoId}	ADMIN	Extrato de comissões e saques de um associado
GET	/api/financeiro/fluxo-caixa	ADMIN	Entradas e saídas mês a mês
GET	/api/financeiro/lancamentos	ADMIN	Lista os lançamentos do período
POST	/api/financeiro/lancamentos	ADMIN	Registra um lançamento manual
DELETE	/api/financeiro/lancamentos/{id}	ADMIN	Remove um lançamento
GET	/api/financeiro/projecao-mrr	ADMIN	Projeção de MRR com base no crescimento recente
GET	/api/financeiro/saques	ADMIN	Lista solicitações de saque
POST	/api/financeiro/saques	autenticado	Solicita um saque das comissões disponíveis
PATCH	/api/financeiro/saques/{id}	ADMIN	Aprova, processa, conclui ou rejeita um saque
GET	/api/financeiro/visao-geral	ADMIN	Saldo, MRR, margem líquida e comissões a pagar

Parceiros e Benefícios · 13 rotas

MÉTODO	CAMINHO	ACESSO	DESCRIÇÃO
GET	/api/beneficios	autenticado	Lista os benefícios
POST	/api/beneficios	ADMIN	Cria um benefício
GET	/api/beneficios/meus-usos	autenticado	Histórico de benefícios resgatados pelo associado logado
DELETE	/api/beneficios/{id}	ADMIN	Remove um benefício
GET	/api/beneficios/{id}	autenticado	Detalha um benefício
PATCH	/api/beneficios/{id}	ADMIN	Atualiza um benefício
POST	/api/beneficios/{id}/resgatar	autenticado	Resgata um benefício
GET	/api/parceiros	ADMIN	Lista parceiros com contagem de benefícios
POST	/api/parceiros	ADMIN	Cadastra um parceiro
GET	/api/parceiros/estatisticas	ADMIN	Totais de parceiros, benefícios e mais utilizados
DELETE	/api/parceiros/{id}	ADMIN	Remove um parceiro sem benefícios vinculados
GET	/api/parceiros/{id}	ADMIN	Detalha um parceiro com seus benefícios
PATCH	/api/parceiros/{id}	ADMIN	Atualiza um parceiro

Conteúdos · 12 rotas

MÉTODO	CAMINHO	ACESSO	DESCRIÇÃO
GET	/api/conteudos	autenticado	Lista os conteúdos
POST	/api/conteudos	ADMIN	Cria um conteúdo

MÉTODO	CAMINHO	ACESSO	DESCRIÇÃO
GET	/api/conteudos/estatisticas	ADMIN	Publicados, rascunhos, visualizações e curtidas
DELETE	/api/conteudos/{id}	ADMIN	Remove um conteúdo
GET	/api/conteudos/{id}	autenticado	Abre um conteúdo por ID ou slug
PATCH	/api/conteudos/{id}	ADMIN	Atualiza um conteúdo
POST	/api/conteudos/{id}/curtir	autenticado	Alterna a curtida do associado no conteúdo
GET	/api/materiais	autenticado	Lista os materiais de divulgação
POST	/api/materiais	ADMIN	Cria um material de divulgação
DELETE	/api/materiais/{id}	ADMIN	Remove um material
PATCH	/api/materiais/{id}	ADMIN	Atualiza um material
GET	/api/materiais/{id}/baixar	autenticado	Registra o download e devolve o arquivo ou o texto

Comunidade · 13 rotas

MÉTODO	CAMINHO	ACESSO	DESCRIÇÃO
GET	/api/comunidade/categorias	autenticado	Lista as categorias com contagem de posts
POST	/api/comunidade/categorias	ADMIN	Cria uma categoria
DELETE	/api/comunidade/comentarios/{id}	autenticado	Remove um comentário (autor ou administrador)
GET	/api/comunidade/estatisticas	autenticado	Posts, comentários, curtidas e membros ativos
GET	/api/comunidade/posts	autenticado	Feed da comunidade
POST	/api/comunidade/posts	autenticado	Publica um post
DELETE	/api/comunidade/posts/{id}	autenticado	Remove um post (autor ou administrador)
GET	/api/comunidade/posts/{id}	autenticado	Abre um post com seus comentários
PATCH	/api/comunidade/posts/{id}	autenticado	Edita o próprio post
POST	/api/comunidade/posts/{id}/comentarios	autenticado	Comenta em um post
POST	/api/comunidade/posts/{id}/curtir	autenticado	Curte ou descurte um post
PATCH	/api/comunidade/posts/{id}/fixar/{valor}	ADMIN	Fixa ou desafixa um post no topo do feed
GET	/api/comunidade/topicos-recentes	autenticado	Tópicos recentes para o dashboard

Suporte · 8 rotas

MÉTODO	CAMINHO	ACESSO	DESCRIÇÃO
GET	/api/suporte	ADMIN	Lista os chamados com filtros e paginação
POST	/api/suporte	autenticado	Abre um chamado
GET	/api/suporte/estatisticas	ADMIN	Abertos, em andamento, resolvidos e tempo médio de resolução

MÉTODO	CAMINHO	ACESSO	DESCRIÇÃO
GET	/api/suporte/meus-chamados	autenticado	Chamados do associado logado
GET	/api/suporte/{id}	autenticado	Abre um chamado por ID ou número (SUP-...)
PATCH	/api/suporte/{id}	ADMIN	Atualiza status, prioridade, categoria ou responsável
PATCH	/api/suporte/{id}/atender	ADMIN	Assume o atendimento do chamado
POST	/api/suporte/{id}/mensagens	autenticado	Responde na thread do chamado

Serviços (Jurídico e Contábil) · 13 rotas

MÉTODO	CAMINHO	ACESSO	DESCRIÇÃO
GET	/api/servicos/escritorios	autenticado	Lista os escritórios parceiros
POST	/api/servicos/escritorios	ADMIN	Cadastra um escritório parceiro
DELETE	/api/servicos/escritorios/{id}	ADMIN	Remove um escritório sem profissionais vinculados
PATCH	/api/servicos/escritorios/{id}	ADMIN	Atualiza um escritório
GET	/api/servicos/estatisticas	autenticado	Escritórios, profissionais e solicitações por status
GET	/api/servicos/minhas-solicitacoes	autenticado	Solicitações do associado logado
GET	/api/servicos/profissionais	autenticado	Lista os profissionais disponíveis
POST	/api/servicos/profissionais	ADMIN	Cadastra um profissional
DELETE	/api/servicos/profissionais/{id}	ADMIN	Remove um profissional
PATCH	/api/servicos/profissionais/{id}	ADMIN	Atualiza um profissional
GET	/api/servicos/solicitacoes	ADMIN	Lista todas as solicitações de atendimento
POST	/api/servicos/solicitacoes	autenticado	Solicita atendimento jurídico ou contábil
PATCH	/api/servicos/solicitacoes/{id}	ADMIN	Designa profissional ou atualiza o status da solicitação

Notificações e Campanhas · 11 rotas

MÉTODO	CAMINHO	ACESSO	DESCRIÇÃO
GET	/api/campanhas	ADMIN	Lista as campanhas com taxa de abertura
POST	/api/campanhas	ADMIN	Cria uma campanha (rascunho ou agendada)
GET	/api/campanhas/estatisticas	ADMIN	Enviadas, agendadas, rascunhos e taxa de abertura geral
PATCH	/api/campanhas/{id}/cancelar	ADMIN	Cancela uma campanha ainda não enviada
POST	/api/campanhas/{id}/enviar	ADMIN	Dispara a campanha para o público-alvo
GET	/api/notificacoes	autenticado	Notificações do associado logado (inclui broadcasts)
POST	/api/notificacoes/enviar	ADMIN	Dispara uma notificação
PATCH	/api/notificacoes/ler-todas	autenticado	Marca todas as notificações como lidas

MÉTODO	CAMINHO	ACESSO	DESCRIÇÃO
GET	/api/notificacoes/resumo	autenticado	Contadores por tipo, canal e não lidas
DELETE	/api/notificacoes/{id}	autenticado	Remove uma notificação
PATCH	/api/notificacoes/{id}/ler	autenticado	Marca uma notificação como lida

Gamificação · 4 rotas

MÉTODO	CAMINHO	ACESSO	DESCRIÇÃO
GET	/api/gamificacao/conquistas	ADMIN	Catálogo de conquistas com quantos associados desbloquearam
GET	/api/gamificacao/minhas-conquistas	autenticado	Conquistas e progresso do associado logado
GET	/api/gamificacao/ranking	autenticado	Ranking com a posição e o percentil do associado logado
POST	/api/gamificacao/recalcular/{associadoId}	ADMIN	Recalcula pontuação e conquistas

Portal do Associado · 22 rotas

MÉTODO	CAMINHO	ACESSO	DESCRIÇÃO
GET	/api/portal/comissoes	autenticado	Comissões do associado logado
GET	/api/portal/cupons	autenticado	Meus cupons com resumo de uso
POST	/api/portal/cupons	autenticado	Cria um cupom de desconto
DELETE	/api/portal/cupons/{id}	autenticado	Remove um cupom ainda não utilizado
PATCH	/api/portal/cupons/{id}	autenticado	Atualiza ou ativa/desativa um cupom
GET	/api/portal/links	autenticado	Meus links com cliques, conversões e receita
POST	/api/portal/links	autenticado	Gera um link rastreável para um produto
GET	/api/portal/loja	autenticado	Minha loja com vitrine e desempenho
PATCH	/api/portal/loja	autenticado	Personaliza a loja
DELETE	/api/portal/loja/produtos/{produtoId}	autenticado	Remove um produto da minha loja
POST	/api/portal/loja/produtos/{produtoId}	autenticado	Adiciona um produto do marketplace à minha loja
GET	/api/portal/perfil	autenticado	Perfil do associado logado
PATCH	/api/portal/perfil	autenticado	Atualiza perfil e preferências de privacidade
GET	/api/portal/performance	autenticado	Cliques, conversões e receita por dia
GET	/api/portal/plano	autenticado	Uso do plano atual, ROI e o que o upgrade desbloqueia
GET	/api/portal/r/{codigo}	PÚBLICO	Redirecionador público do link de afiliado
GET	/api/portal/redes-sociais	autenticado	Contas conectadas e métricas agregadas

MÉTODO	CAMINHO	ACESSO	DESCRIÇÃO
POST	/api/portal/redes-sociais	autenticado	Conecta ou atualiza uma rede social
DELETE	/api/portal/redes-sociais/{rede}	autenticado	Desconecta uma rede social
GET	/api/portal/saldo	autenticado	Saldo de comissões do associado logado
GET	/api/portal/saques	autenticado	Saques do associado logado
GET	/api/portal/vendas	autenticado	Vendas do associado logado

A interface

O site tem três faces com públicos e permissões distintos: o painel administrativo, o portal do associado e as páginas abertas. São **40 telas** em React, todas consumindo a mesma API.

8.1 Tecnologias e por que cada uma

PEÇA	VERSÃO	PAPEL
React	19	Base da interface. Escolhido também por continuidade: a plataforma original era React.
Vite	8	Servidor de desenvolvimento com recarga instantânea e empacotador de produção. Faz proxy de <code>/api</code> , o que elimina CORS no desenvolvimento.
TypeScript	5	Os contratos da API são declarados em <code>api/tipos.ts</code> ; mudar um campo no backend quebra a compilação do front, em vez de falhar em produção.
Ant Design	5	Biblioteca de componentes. Mesma do site original, o que preserva a ergonomia das telas capturadas — tabelas, formulários e feedback já resolvidos.
TanStack Query	5	Cache, revalidação e estados de carregamento. Uma escrita invalida as chaves afetadas e a tela se atualiza sozinha, sem recarregar nada à mão.
Recharts	2	Gráficos em SVG. Também era a biblioteca do original.
Axios	1	Cliente HTTP com interceptadores — é onde vive a renovação automática de token.

8.2 As três áreas

Painel administrativo — 22 telas

Dashboard com oito indicadores e quatro gráficos; listagens de associados, catálogo, lojas, vendas, comissões, parceiros, benefícios, conteúdos e chamados; telas de detalhe para associado, produto, loja e ticket; formulários de cadastro; e as áreas analíticas de financeiro (fluxo de caixa, DRE, projeção de MRR, saques) e relatórios.

Portal do associado — 18 telas

Painel pessoal, minha loja (com marketplace para escolher produtos), minhas vendas, financeiro com solicitação de saque, cupons, links de afiliado, performance diária, benefícios, conteúdos, materiais de divulgação, comunidade, ranking e conquistas, assessoria jurídica e contábil, suporte, redes sociais, meu plano e meu perfil.

Páginas públicas — 2 telas

A vitrine da loja (`/loja/:slug`) e o perfil do criador (`/perfil/:handle`), ambas sem exigir login, mais o gate de código de acesso que precede a tela de entrada.

8.3 Duas decisões que moldam o resto

FILTRO NO SERVIDOR, NUNCA NO CLIENTE

O componente `TabelaRecurso` envia busca, filtros, ordenação e paginação como parâmetros de consulta. Filtrar no navegador só alcançaria a página já carregada — buscar “ana” varreria 10 registros em vez dos 40. Como consequência, a mesma tabela serve a qualquer volume de dados sem mudar de estratégia.

RENOVAÇÃO DE TOKEN COM CORRIDA ÚNICA

Ao receber `401` , o cliente renova o token e repete a requisição. Chamadas simultâneas que falhem no mesmo instante aguardam a *mesma* renovação: o refresh token é rotativo e só vale uma vez, então dispará-las em paralelo invalidaria a sessão do próprio usuário.

8.4 Identidade visual

A interface segue o brand book oficial. As cores não foram convertidas de tabelas Pantone de memória: foram **amostradas dos pixels** das cartelas do próprio PDF. O *Indigo Sloth* amostrado coincidiu exatamente com o `#230647` que o documento declara — o que valida a extração das outras três.

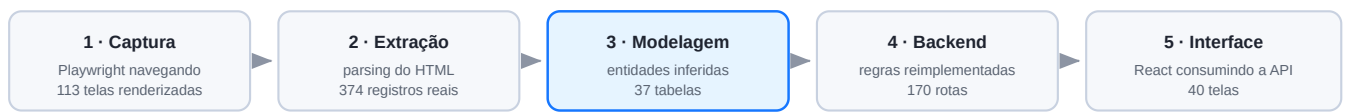
COR	HEX	PANTONE	ONDE APARECE
Digital Blue	<code>#008EEA</code>	2727 C	primária — botões, links, série principal dos gráficos
Mint Leaf	<code>#00AD9A</code>	3275 C	positivos — receita, comissões pagas, confirmações
Violet C	<code>#440099</code>	Violet C	acentos e segunda série dos gráficos
Indigo Sloth	<code>#230647</code>	—	fundos institucionais (acesso e login)

A tipografia do brand book — *Bebas Kai* (títulos) e *Lufga* (texto) — é comercial. A pilha em `src/marca.ts` declara as originais primeiro e recai sobre *Bebas Neue* e *Poppins*, de modo que uma instalação licenciada passa a valer automaticamente, sem tocar no código. Os logos vieram do kit oficial, recortados com fundo transparente para servirem sobre claro e escuro; o favicon sai do símbolo da digital.

Como foi desenvolvido

Este sistema não partiu de uma especificação escrita. Ele foi reconstruído por engenharia reversa a partir de 113 telas capturadas da plataforma original — uma SPA React com Firebase, cujo bundle minificado não permitia recuperar o código-fonte.

9.1 As cinco etapas



Do HTML capturado ao sistema em funcionamento.

A etapa 3 é a que decide tudo. Uma coluna “Comissão” numa tabela e uma coluna “%” em outra revelam que comissão é entidade própria, com percentual e valor. Um badge “Intermediário+” num produto revela o campo de exclusividade por plano. Um contador “47 / 100” num cupom revela usos e limite. O schema saiu dessa leitura, não de suposição.

9.2 O elo que ficou reexecutável

`tools/extract.py` percorre as capturas e produz 24 arquivos JSON que alimentam o seed. Ele não foi um script descartável: continua rodando e é o que garante que os dados do banco sejam os reais das telas, não invenção.

```
python3 tools/extract.py          # capturas -> prisma/seed-data/*.json
cd apps/api && npm run seed       # JSON -> PostgreSQL
```

9.3 O que a conferência de números revelou

Comparar os indicadores calculados com os das telas originais funcionou como teste de modelagem, e expôs dois erros que passariam despercebidos:

SINTOMA	CAUSA	CORREÇÃO
MRR dava R\$ 3.497 em vez de R\$ 1.998	Assinatura de associado suspenso ou inativo contava como vigente	<code>Assinatura.ativa</code> passou a acompanhar o status do associado
Saldo financeiro não fechava	O caixa fora modelado como meses acumulados de MRR	Entradas = receita das vendas pagas; saídas = comissões pagas + custos

Depois das correções, treze indicadores coincidem com a fonte — receita, ticket médio, comissões em cada estado, MRR e a composição percentual das saídas do DRE. O detalhamento está no capítulo 12.

Escalabilidade e capacidade

Este capítulo traz **medições**, não estimativas. Todos os números vieram de testes de carga executados com `autocannon` contra a API em modo de produção, com o banco populado.

O QUE ESTES NÚMEROS SÃO E O QUE NÃO SÃO

Foram medidos em uma máquina de desenvolvimento (AMD Ryzen AI 9 HX 370, 24 núcleos, 93 GB de RAM), com API e banco no mesmo host e sem latência de rede. Servem para comparar endpoints entre si e revelar gargalos — **não** como promessa de desempenho em produção, onde entram rede, disco compartilhado e recursos menores. Um teste no ambiente real de destino continua necessário.

10.1 Capacidade por endpoint

Cinquenta conexões simultâneas, 12 segundos por endpoint, após aquecimento:

ENDPOINT	REQ/S	P50	P90	P99	PERFIL
GET /planos	1.906	25 ms	29 ms	40 ms	leitura leve
GET /vendas	1.174	38 ms	44 ms	59 ms	lista paginada
GET /financeiro/visao-geral	1.244	33 ms	56 ms	81 ms	6 agregações
GET /comunidade/posts	1.042	44 ms	65 ms	81 ms	lista com contagens
GET /catalogo/produtos	1.068	42 ms	62 ms	86 ms	lista com joins
GET /associados	868	55 ms	80 ms	104 ms	joins + subconsultas
GET /relatorios	754	62 ms	89 ms	140 ms	analítico pesado
GET /lojas/publica/:slug	667	65 ms	117 ms	197 ms	público + escrita de contador
GET /dashboard/admin	658	74 ms	—	115 ms	18 agregações paralelas
POST /auth/login	88	560 ms	—	622 ms	bcrypt, por natureza caro

10.2 Comportamento sob concorrência

Mesmo endpoint (`/associados`), concorrência crescente. O throughput satura por volta de 100 conexões; além disso a latência cresce linearmente sem ganho de vazão — a fila apenas aumenta.

CONEXÕES	REQ/S	P50	P99	LEITURA
10	935	7 ms	19 ms	folgado
25	1.077	22 ms	41 ms	confortável

CONEXÕES	REQ/S	P50	P99	LEITURA
50	1.185	40 ms	58 ms	bom
100	1.243	78 ms	100 ms	joelho da curva
200	1.214	160 ms	221 ms	saturado — só enfileira
400	1.111	334 ms	482 ms	saturado

10.3 Quantos usuários simultâneos

“Conexões simultâneas” não é “usuários simultâneos”. Uma pessoa navegando não mantém uma requisição contínua: ela carrega uma tela, lê, clica de novo. Com um *tempo de leitura* típico de 8 a 12 segundos entre ações, e considerando que cada tela dispara em média 3 requisições, um usuário gera cerca de **0,3 requisição por segundo**.

~2.500 USUÁRIOS ATIVOS navegação típica, folga confortável	~4.000 NO LIMITE latência ainda aceitável	88/s LOGINS o verdadeiro teto em picos
---	--	---

Com throughput sustentado de ~1.200 req/s, a conta é $1.200 \div 0,3 \approx 4.000$ usuários no limite, e algo perto de **2.500 com folga** — mantendo p99 abaixo de 100 ms. *Uma única instância Node*, note-se, usando um dos 24 núcleos.

O LOGIN É O GARGALO EM PICOS

A 88 logins por segundo, um evento que faça 5.000 pessoas entrarem ao mesmo tempo leva cerca de **57 segundos** para escoar a fila. Isso é intrínseco ao bcrypt, que é lento de propósito para resistir a ataques de força bruta — reduzir o custo enfraqueceria as senhas. A saída correta é distribuir a carga entre processos (seção 10.5), não baratear o hash.

10.4 O gargalo que foi corrigido durante a medição

A primeira rodada mediu **17–20 logins por segundo**, com p50 de 2,5 segundos, e — pior — as leituras *também* ficavam lentas durante o login. A causa era o `bcryptjs`, implementação em JavaScript puro que ocupa a thread principal: enquanto calculava um hash, o event loop parava para todos os demais pedidos.

A troca pelo `@node-rs/bcrypt` (binário nativo, que roda no threadpool do libuv) resolveu os dois problemas de uma vez. Os hashes são compatíveis, então nenhuma senha precisou ser redefinida.

MEDIDA	BCRYPTJS	NATIVO	GANHO
Logins por segundo	20	88	4,4×
p50 do login (50 conexões)	2.577 ms	560 ms	4,6×
<code>/planos</code> durante carga de login	degradado	1.272 req/s, p50 16 ms	sem bloqueio

10.5 Caminhos de crescimento

A arquitetura não guarda estado em memória — a sessão vive no token e no banco. Isso é o que torna a escala horizontal possível sem reescrever nada.

#	PASSO	GANHO ESPERADO	CUSTO
1	Clusterizar o Node (PM2 ou modo cluster) com 4–8 workers	4–8× o throughput e os logins	Baixo — uma linha de configuração. É o passo de melhor retorno.
2	Cache nos analíticos — dashboard e relatórios, 30–60 s	Tira as consultas mais caras do caminho quente	Baixo — Redis ou cache em memória por worker.
3	Ajustar o pool do Prisma ao número de workers	Evita esgotar as 100 conexões do Postgres	Baixo — <code>connection_limit</code> na URL do banco.
4	Réplica de leitura no Postgres para os relatórios	Isola analítico de transacional	Médio — exige roteamento de consultas.
5	Várias instâncias atrás de um balanceador	Escala praticamente linear	Médio — o código já suporta; é infraestrutura.
6	CDN para o front	Tira todo o estático da API	Baixo — o build do Vite é estático puro.

Só com o passo 1, a projeção sobe para a faixa de **15.000 a 20.000 usuários ativos** na mesma máquina, com o login saindo de 88 para algo em torno de 500 por segundo.

10.6 Onde o banco vai apertar primeiro

TABELA	CRESCE COM	OBSERVAÇÃO
<code>metricas_diarias</code>	associados × dias	10 mil associados × 365 dias = 3,6 M linhas/ano. Já pré-agregada e indexada; particionar por ano quando incomodar.
<code>notificacoes</code>	associados × eventos	A que mais cresce. Precisa de política de expurgo — arquivar o que passou de 90 dias.
<code>vendas</code> e <code>comissoes</code>	volume transacional	Crescimento saudável; os índices por status e data já cobrem as consultas usadas.
<code>logs_auditoria</code>	toda operação sensível	Também exige expurgo ou envio para armazenamento frio.

Análise de segurança

Esta análise combina revisão do código com **testes ofensivos executados contra a API em funcionamento** — tentativas reais de ler dados alheios, escalar privilégio e reutilizar tokens.

11.1 Testes de autorização executados

TENTATIVA	RESULTADO	VEREDITO
Associado altera cupom de <i>outro</i> associado	403	✔ bloqueado
Associado abre chamado de suporte de outro	403	✔ bloqueado
Associado força <code>associadoId</code> alheio na consulta	400	✔ parâmetro nem é aceito
Associado lista todos os associados	403	✔ bloqueado
Associado cria produto no catálogo	403	✔ bloqueado
Associado liquida comissões	403	✔ bloqueado
Associado acessa o financeiro global	403	✔ bloqueado
Token adulterado	401	✔ assinatura verificada
Requisição sem token	401	✔ bloqueado
Refresh token usado como access token	401	✔ tipos não se confundem
Reutilização de refresh token já consumido	401	✔ rotação funciona

11.2 Controles implementados

CONTROLE	COMO ESTÁ
Senhas	bcrypt com 10 rounds, implementação nativa. O hash nunca sai da camada de serviço — nenhuma rota o inclui na resposta.
Refresh token	Guardado como SHA-256, com rotação a cada uso. Um token roubado só serve uma vez, e o uso legítimo seguinte falha — o que denuncia o roubo.
Access token	Válido por 15 minutos. O usuário é revalidado no banco a cada requisição, então desativar uma conta tem efeito imediato.
Injeção de SQL	Prisma parametriza tudo. Os dois <code>\$queryRaw</code> usam template tag, que também parametriza. O único <code>\$executeRawUnsafe</code> está no seed, com SQL fixo e sem entrada de usuário.
Mass assignment	<code>forbidNonWhitelisted</code> rejeita com 400 qualquer campo não declarado no DTO — foi o que barrou a tentativa de forçar <code>associadoId</code> .
Autorização horizontal	Verificada no serviço, comparando o dono do recurso com o associado autenticado. As rotas do portal impõem o filtro no servidor.

CONTROLE	COMO ESTÁ
Confidencialidade no suporte	Notas internas são filtradas da resposta quando quem consulta não é administrador.
Rate limiting	300 requisições por minuto por IP.
Cabeçalhos	Helmet aplicado.
Enumeração de contas	Mensagem idêntica para email inexistente e senha errada — e tempo de resposta equalizado (ver 11.3).

11.3 Vulnerabilidade encontrada e corrigida

ENUMERAÇÃO DE USUÁRIOS POR TEMPORIZAÇÃO

A mensagem de erro já era idêntica nos dois casos, mas o **tempo** não era. Quando o email não existia, o código retornava de imediato (6,7 ms); quando existia, rodava o bcrypt antes de falhar (51,1 ms). A diferença de 44 ms permitia a um atacante descobrir, por cronometragem, quais contas existem — insumo para ataques direcionados de senha.

Correção: a comparação passou a rodar sempre, contra um hash-isca gerado a cada inicialização quando o email não existe. A diferença caiu para **3,4 ms**, dentro do ruído da rede.

11.4 Pendências antes de expor à internet

RISCO	ITEM	POR QUÊ
Alto	Segredos ainda são os de exemplo	<code>JWT_SECRET</code> e <code>JWT_REFRESH_SECRET</code> continuam com o valor do <code>.env.example</code> , que está no repositório. Quem o lê forja qualquer token. <i>Trocar é obrigatório.</i>
Alto	Senhas do seed publicadas	<code>Admin@2026</code> está nesta documentação e no repositório.
Alto	Sem HTTPS	O Bearer token viaja no cabeçalho; sem TLS é interceptável na rede.
Médio	<code>CORS_ORIGINS</code> aceita <code>*</code> como padrão	Se a variável não for definida em produção, qualquer origem chama a API.
Médio	Rate limit único para todas as rotas	300/min é generoso para <code>/auth/login</code> . Um limite específico e mais estrito no login dificultaria força bruta.
Médio	Sem bloqueio após tentativas falhas	Não há travamento temporário de conta nem captcha.
Baixo	CSP desativada no Helmet	<code>contentSecurityPolicy: false</code> foi posto para não atrapalhar o Swagger. Convém ativar em produção, ou publicar o Swagger em rota separada.
Baixo	Sem rotação de logs de auditoria	A tabela cresce sem limite.

RISCO	ITEM	POR QUÊ
Info	Gate de código de acesso não é segurança	Reproduz o comportamento original, mas é um código compartilhado — trata-se de uma barreira de conveniência, não de um controle de acesso.

Dependências

O `npm audit` aponta 3 vulnerabilidades de severidade alta, **todas em dependências de desenvolvimento** (o parser de configuração da CLI do Prisma e o gerador de identificadores do autocannon). Nenhuma alcança o código que roda em produção. Ainda assim, vale acompanhar as atualizações.

Dados e fidelidade

O banco não é populado com dados sintéticos. O seed carrega o dataset extraído das telas reais da plataforma original — e os indicadores calculados pela API reproduzem os números que aquelas telas exibiam.

12.1 O pipeline de extração

`tools/extract.py` percorre as 113 capturas e produz 24 arquivos JSON. Ele opera em duas frentes: as **tabelas** (parsing das linhas `<tr>` com o identificador real no atributo `data-row-key`) e as **telas em cartão** — planos, benefícios, conteúdos, notificações, serviços, materiais, conquistas e comunidade — cujos dados vivem em estruturas visuais do Ant Design.

```
cd digitaisbr-clone
python3 tools/extract.py      # regenera prisma/seed-data/*.json
cd apps/api && npm run seed   # recarrega o banco
```

12.2 O que o seed carrega

374REGISTROS
EXTRAÍDOS**24**

ARQUIVOS JSON

37TABELAS
POPULADAS**~2s**

TEMPO DE CARGA

ENTIDADE	QTD.	ENTIDADE	QTD.
Planos	3	Benefícios	19
Associados	40 ¹	Conteúdos	16
Produtos / categorias	25 / 7	Materiais de divulgação	12
Lojas / itens de vitrine	20 / 150	Posts / comentários / curtidas	20 / 349 / 498 ²
Vendas	60	Tickets / mensagens	21 / 82
Comissões	40	Escritórios / profissionais	5 / 6
Parceiros	10	Notificações / campanhas	25 / 5
Cupons / links / saques	5 / 5 / 8	Conquistas	12

¹ São 30 associados da tabela original mais 10 personas que apareciam apenas na comunidade e no suporte (Maria Silva, João Santos, Ana Oliveira e outros). Elas foram criadas como associados para preservar a integridade referencial daquele conteúdo — a alternativa seria descartar os posts e chamados delas.

² O dataset registrava 576 curtidas, mas a curtida é única por (post, pessoa) e há 40 associados. O total possível é 498. Os comentários (349) coincidiram exatamente.

12.3 Conferência com as telas originais

Esta tabela é a prova de que o modelo está certo: são indicadores calculados por agregação SQL sobre o banco, comparados com o que a plataforma original exibia.

INDICADOR	TELA ORIGINAL	API	CONFERE
Receita total	R\$ 4.395,00	R\$ 4.395,00	✓
Ticket médio	R\$ 146,50	R\$ 146,50	✓
Vendas aprovadas	30	30	✓
Taxa de cancelamento	33,3%	33,3%	✓
Comissões pagas	R\$ 373,02	R\$ 373,02	✓
Comissões em processamento	R\$ 561,45	R\$ 561,45	✓
Comissões aguardando	R\$ 350,92	R\$ 350,92	✓
MRR	R\$ 1.998,00	R\$ 1.998,00	✓
Saldo financeiro	R\$ 3.378,46	R\$ 3.378,50	≈ ³
Margem líquida	76,8%	76,9%	≈ ³
Composição de saídas	36,7 / 23,7 / 19,8 / 11,9 / 7,9 %	idêntica	✓
Comentários na comunidade	349	349	✓
Visualizações de conteúdo	44.690	44.690	✓

³ Diferença de R\$ 0,04, decorrente do arredondamento a dois decimais ao ratear os custos operacionais entre os meses de competência.

DUAS CORREÇÕES QUE ESSES NÚMEROS REVELARAM

O MRR saía R\$ 3.497 até se perceber que assinatura de associado suspenso ou inativo não pode contar como vigente — `Assinatura.ativa` passou a acompanhar o status do associado. E o saldo só fechou quando o financeiro foi modelado como a plataforma o modelava: *entradas = receita das vendas pagas e saídas = comissões pagas + custos operacionais*, em vez de meses acumulados de MRR. Conferir com a fonte é o que expõe esse tipo de erro.

Qualidade e testes

A suíte end-to-end exercita a API pela porta HTTP, contra um banco real com o seed carregado. Ela não usa mocks: se uma transação estiver quebrada, o teste quebra.

77

ASSERÇÕES

todas passando

2

SUÍTES

leitura e escrita

0

MOCKS

banco e HTTP reais

13.1 O que a suíte cobre

BLOCO	VERIFICAÇÕES
Autenticação	Rota protegida sem token devolve 401; login de admin e de associado; senha errada rejeitada; código de acesso validado.
RBAC	Associado bloqueado nas rotas administrativas; admin sem associado bloqueado no portal; cada papel acessa o que lhe cabe.
Integridade do seed	Contagem exata de cada entidade carregada.
Listagens	Paginação, total de páginas, busca textual, filtro por status e plano, ordenação e recorte por período.
Validação	Email malformatado, campo desconhecido, UUID inválido, enum inexistente e limite acima do máximo — todos rejeitados com 400.
Ciclo da venda	Total calculado, comissão gerada com o percentual correto, referência sequencial, transições válidas aceitas, inválidas recusadas, estados finais fechados, e a propagação para a comissão.
Cupons	Criação, código duplicado recusado, desconto aplicado ao total, cupom inexistente recusado.
Comissões e saques	Liquidação em lote, repagamento recusado, saque acima do saldo recusado.
Limites por plano	Limite de produtos do Básico, personalização visual bloqueada, produto exclusivo recusado, e o que é permitido no Básico continua permitido.
Comunidade	Publicar, curtir, descurtir, comentar, fixar como admin e a recusa ao associado.
Suporte	Numeração, thread inicial, nota interna recusada ao associado, transição automática de status, e a contagem de mensagens visíveis a cada papel.
Isolamento	Chamado de outro associado devolve 403.
Rotas públicas	Vitrine e perfil sem autenticação, privacidade de email respeitada e contabilização de visualização.

13.2 Como executar

```
cd apps/api
npm run start:prod &      # a API precisa estar no ar
npm run seed              # estado limpo e determinístico
npm run test:e2e
```

POR QUE RESEEDAR ANTES

A suíte de escrita cria vendas, cupons, posts e chamados reais. Rodá-la duas vezes sem reseedar faz a segunda execução esbarrar nos dados da primeira — um cupom com código duplicado, por exemplo. Isso não é fragilidade do teste: é a API recusando corretamente uma operação inválida.

Operação

14.1 Subir o ambiente

```
# 1. banco – Postgres em :5434 e Adminer em :8082
docker compose up -d

# 2. dependências e variáveis
cd apps/api
npm install
cp .env.example .env

# 3. schema e dados
npx prisma migrate deploy
npm run seed

# 4. subir a API
npm run start:dev
```

SERVIÇO	ENDEREÇO	OBSERVAÇÃO
API	localhost:3000/api	—
Swagger	localhost:3000/api/docs	Documentação navegável e testável
PostgreSQL	localhost:5434	Porta 5432 evitada por conflito com outros contêineres
Adminer	localhost:8082	Servidor db , usuário e senha digitaisbr

Credenciais do seed

PERFIL	EMAIL	SENHA
Administrador	administrador@digitaisbr.com	Admin@2026
Associado	ana-silva@email.com	Assoc@2026

Os 40 associados usam a mesma senha, com email no padrão handle@email.com .

14.2 Variáveis de ambiente

VARIÁVEL	EXEMPLO	FUNÇÃO
DATABASE_URL	postgresql://...:5434/digitaisbr	Conexão com o Postgres
PORT	3000	Porta HTTP
API_PREFIX	api	Prefixo global das rotas

VARIÁVEL	EXEMPLO	FUNÇÃO
<code>CORS_ORIGINS</code>	<code>http://localhost:5173</code>	Origens liberadas, separadas por vírgula
<code>JWT_SECRET</code>	—	Assinatura do access token
<code>JWT_EXPIRES_IN</code>	<code>15m</code>	Validade do access token
<code>JWT_REFRESH_SECRET</code>	—	Segredo do refresh
<code>JWT_REFRESH_EXPIRES_IN</code>	<code>7d</code>	Validade do refresh
<code>ACCESS_CODE</code>	—	Código do gate de acesso da plataforma
<code>SEED_ADMIN_EMAIL</code> <code>SEED_ADMIN_PASSWORD</code>	—	Credenciais criadas pelo seed
<code>SEED_ASSOCIADO_PASSWORD</code>	—	Senha padrão dos associados do seed

14.3 Migrations

```

npx prisma migrate dev --name descricao # cria e aplica em desenvolvimento
npx prisma migrate deploy               # aplica em produção, sem prompts
npx prisma migrate reset --force        # recria do zero e roda o seed
npx prisma studio                      # inspeção visual do banco
npx prisma generate                    # regenera o cliente tipado

```

14.4 Checklist antes de produção

ITEM	POR QUÊ
☐ Trocar <code>JWT_SECRET</code> e <code>JWT_REFRESH_SECRET</code>	Os valores do <code>.env.example</code> são públicos. Use segredos aleatórios de 32+ bytes.
☐ Alterar as senhas do seed	<code>Admin@2026</code> está nesta documentação e no repositório.
☐ Restringir <code>CORS_ORIGINS</code>	Listar apenas os domínios do front, sem curinga.
☐ Servir sob HTTPS	O Bearer token trafega no cabeçalho; sem TLS ele é interceptável.
☐ Rever o limite do throttler	300 req/min por IP serve para desenvolvimento; ajuste ao tráfego real.
☐ Configurar backup do Postgres	O volume Docker não é backup.
☐ Rodar <code>migrate deploy</code> , nunca <code>migrate dev</code>	<code>dev</code> pode propor reset destrutivo.
☐ Não rodar o seed em produção	O seed trunca todas as tabelas antes de carregar.
☐ Centralizar logs e monitorar 5xx	Os filtros já registram stack trace de erro de servidor.

DUAS PENDÊNCIAS CONHECIDAS

Envio real de email e push — notificações e campanhas são gravadas no banco e servidas pela API, mas nenhum provedor externo é acionado. Integrar um serviço de envio é trabalho pendente.

Upload de arquivos — imagens de produto, banner e logo são recebidas como URL. Não há armazenamento de binários; conectar um bucket é o passo natural.

Publicar na internet

O que existe hoje roda em `localhost`, com dados de demonstração e segredos de exemplo. Este capítulo lista o que falta para colocar o sistema no ar de forma segura, em ordem de dependência.

15.1 Arquitetura de produção



Topologia mínima recomendada. Nada aqui exige mudança no código da aplicação.

15.2 Checklist obrigatório — sem isto, não publique

ITEM	COMO FAZER
☐ Gerar segredos novos	O <code>.env.example</code> está no repositório e seus valores são públicos. <code>openssl rand -base64 48</code> para cada um de <code>JWT_SECRET</code> e <code>JWT_REFRESH_SECRET</code> .
☐ Trocar as senhas do seed	<code>Admin@2026</code> e <code>Assoc@2026</code> estão publicadas nesta documentação. Crie o administrador real e remova as contas de demonstração.
☐ TLS em tudo	Let's Encrypt via Caddy (automático) ou Certbot com Nginx. Redirecionar <code>80</code> → <code>443</code> .
☐ Restringir CORS	<code>CORS_ORIGINS=https://digitaisbr.com</code> . O padrão atual é <code>*</code> .
☐ Não rodar o seed	Ele executa <code>TRUNCATE</code> em todas as tabelas antes de carregar.
☐ Usar <code>migrate deploy</code>	Nunca <code>migrate dev</code> em produção — ele pode propor reset destrutivo.
☐ Backup do banco	Volume Docker não é backup. Use um Postgres gerenciado com snapshot diário, ou <code>pg_dump</code> agendado para armazenamento externo.
☐ Senha forte no Postgres	Hoje é <code>digitaisbr/digitaisbr</code> . O banco não deve aceitar conexão externa.

15.3 Checklist recomendado

ITEM	MOTIVO
☐ Cluster de 4 a 8 workers	Multiplica o throughput e, principalmente, a capacidade de login (seção 10.5).
☐ Rate limit específico no login	Algo como 10 tentativas por minuto por IP, contra força bruta.
☐ Logs centralizados e alerta de 5xx	Os filtros já registram stack trace; falta para onde enviar.
☐ Ativar CSP no Helmet	Está desligada por causa do Swagger — publique a documentação em rota separada ou restrita.
☐ Swagger fechado em produção	Expor o mapa completo da API facilita reconhecimento. Restrinja por IP ou desative.
☐ Cache nos analíticos	Dashboard e relatórios são as consultas mais caras.
☐ Expurgo de notificações e auditoria	São as tabelas que crescem sem teto.
☐ Healthcheck e reinício automático	<code>GET /api/planos</code> serve como sonda; PM2 ou Docker reiniciam em falha.

15.4 Funcionalidades ainda não implementadas

Dois recursos são recebidos e armazenados pelo sistema, mas não têm integração externa:

RECURSO	SITUAÇÃO	O QUE FALTA
Email e push	Notificações e campanhas são gravadas no banco e servidas pela API; nada sai para fora.	Integrar um provedor de envio (Resend, SendGrid, SES) e um serviço de push. O modelo de dados já prevê o canal.
Upload de arquivos	Imagens de produto, banner e logo são recebidas como URL.	Conectar um bucket S3 ou R2 e trocar os campos de URL por upload com validação de tipo e tamanho.

15.5 Passo a passo do deploy

```
# 1. servidor: Docker + proxy reverso
apt update && apt install -y docker.io docker-compose-plugin caddy

# 2. variáveis de produção (fora do repositório)
JWT_SECRET=$(openssl rand -base64 48)
JWT_REFRESH_SECRET=$(openssl rand -base64 48)
DATABASE_URL="postgres://usuario:SENHA_FORTE@db:5432/digitaisbr?connection_limit=20"
CORS_ORIGINS="https://digitaisbr.com"
NODE_ENV=production

# 3. banco: aplicar migrations (nunca o seed)
npx prisma migrate deploy

# 4. API em cluster
npm run build
pm2 start dist/main.js -i 4 --name digitaisbr-api

# 5. front: build estático
cd apps/web && npm run build      # gera dist/
```

Configuração mínima do Caddy — TLS é automático:

```
digitaisbr.com {
  # a API responde sob /api
  handle /api/* {
    reverse_proxy localhost:3000
  }
  # todo o resto é o front; fallback para o index (rotas do SPA)
  handle {
    root * /var/www/digitaisbr/dist
    try_files {path} /index.html
    file_server
  }
}
```

15.6 Estimativa de infraestrutura

PORTE	USUÁRIOS ATIVOS	SERVIDOR	BANCO
Piloto	até 500	2 vCPU · 4 GB — API e banco juntos	Postgres no mesmo host
Produção inicial	2.000 – 5.000	4 vCPU · 8 GB — 4 workers	Gerenciado, 2 vCPU · 4 GB, com backup
Crescimento	15.000 – 20.000	8 vCPU · 16 GB — 8 workers, ou 2 instâncias	Gerenciado, 4 vCPU · 8 GB + réplica de leitura

Projeção derivada das medições do capítulo 10, assumindo navegação típica (uma ação a cada 8–12 segundos). Vale como ponto de partida para dimensionar, não como garantia — confirme com um teste de carga no ambiente real antes de abrir ao público.